GSoC'25 Proposal: Lunatik HID Framework Implementation

Jieming Zhou

This proposal introduces myself and outlines the work I will be doing during the Google Summer of Code 2025.
My proposal aims to implement a HID driver framework for Lunatik, a module that provides a way to binding HID devices using lua script.

March 24, 2025

Contents

1. My GSoC'25 Résumé 3
   1.1. Basic information 3
   1.2. Skills 3
   1.3. Experience in Team Work 4
      1.3.1. Internship 4
         1.3.1.1. Intern at Tiktok(China) 4
      1.3.2. Project Experience 4
         1.3.2.1. Bot DSL in QQ Chat 4
      1.3.3. Open Source Contribution 4
   1.4. GSOC 4
2. Project Proposal 5
   2.1. Reason for Choosing it 5
   2.2. Project Design 5
      2.2.1. Background 5
      2.2.2. Architecture Design 6
      2.2.3. API Function Design 7
   2.3. Expected Deliverables 8
      2.3.1. Basic Module luahid 8
      2.3.2. Input Module luahid.input 10
      2.3.3. Other Useful functions 12
      2.3.4. Plan to Port a HID Driver 13
      2.3.5. Benchmarks 13
   2.4. Timeline and Milestones 14
3. Post GSoC 18

Bibliography 19

Index of Figures 20

1. My GSoC'25 Résumé

Here is a brief overview of my skills and experience that make me a suitable candidate for the Google Summer of Code 2025.

1.1. Basic information

Name: Jieming Zhou

Email: qrsikno@gmail.com°

Github https://github.com/qrsikno2°

Matrix @qrsikno:matrix.org(nickname: ketsumin chow)

Timezone: `UTC+8` , China

Academic Background: Major in **Computer Science**, Beijing University of Posts and Telecommunications, Bachelor's degree candidate, expected to graduate in June 2025.

Other time Commitments: I have no other commitments during the GSoC period.

1.2. Skills

I have experience with C/C++, Lua, Python, Golang, and some other languages. I have a basic understanding of the following topics:

1. **Operating System**: Linux, Windows
2. **Backend Development**: Golang, Python
3. **Tools for Development**: Qemu, GDB, GNU Make, NeoVim
4. **Version Control**: Git, Github
5. **Live Streaming**: OBS, FFmpeg
6. **Markup Language**: Markdown, Typst

About my previous experience with Lua:

> I have researched in real-time streaming systems for live broadcasting platforms, where I leveraged Lua VMs to manage dynamic QoS policies. And I have used Lua in game strategy mod optimization and my NeoVim configure.
>
> I have practical experience with Lua C APIs in my past projects, such as embedding Lua in C/C++ applications, and I have a basic understanding of the Lua C API.

1.3. Experience in Team Work

1.3.1. Internship

1.3.1.1. Intern at Tiktok(China)

LLM Backend Dev Intern @Tiktok(China, aka Bytedance)

Language: Golang, Python

2024.10-2025.2

- Worked as a development team member in Tiktok(China) Machine Learning System Team which is used by all model training tasks, average about 1000~2000 DAUs.
- Enhanced Machine Learning dataset synchronization from China to rest of the world in Tiktok.
- Designed a mechanism & framework about Precheck of LLM PreTrain Tasks.
- Refined Training/Evaluation Data Management with Data Lakes such as an improved version of Apache Iceberg.
- Enhanced LLM Evaluation System in its ETL(Extract-Transform-Load) by python.

1.3.2. Project Experience

1.3.2.1. Bot DSL in QQ Chat

I'm a main contributor of a Bot DSL project in QQ chat, which is used to control the Bot's behavior.

Designed a simple DSL for bot control in QQ Chat(China's popular instant messaging platform).

Language: Python

2023.10-2024.1

- Designed a simple DSL and its grammar.
- Implemented Language Virtual Machine and a REPL environment for the DSL.
- Integrated Integrated specialized LLM APIs in the DSL.

1.3.3. Open Source Contribution

While I have not yet contributed to open-source projects, I am excited to make Lunatik my first open-source contribution. 😄

1.4. GSOC

This is my first year of GSoC, and I have no other apply to any other organizations. I am very interested in the Lunatik project and I am looking forward to contributing to it. 🆒

2. Project Proposal

I'm going to apply for "Lunatik binding for Human Interface Devices (HID) drivers" project.

2.1. Reason for Choosing it

1. I'm very interested in Lua and Linux device driver development, so Lunatik is a perfect project for me to learn and contribute.
2. I have some experience in C Language Systematic Programming. And from my point of view, I find Lunatik is a powerful project in processing Fast and Flexible software needs.
3. Developing HID drivers presents both opportunities and technical challenges that will enhance my ability to analyze and contribute to the Linux kernel codebase. This project would significantly improve my skills in working with low-level system programming while addressing real-world needs in device driver development.
4. This is my first year of GSoC, and I want to contribute to the open-source community. I think Lunatik is a good start for me.

2.2. Project Design

2.2.1. Background

> Lunatik binding for Human Interface Devices (HID) drivers
>
> Lunatik is a framework for scripting the Linux kernel with Lua. For example, Lunatik can be used for scripting the Linux networking subsystem (as presented at Netdev 0x14 and 0x17) among other examples.
>
> The purpose of this project is to create a Lunatik library for binding the Linux HID APIs to allow developers to write new HID drivers using Lua. This project might leverage the Lunatik device library, which allows the creation of character device drivers using Lua. Moreover, this project should also port at least one HID driver to Lua.

2.2.2. Architecture Design

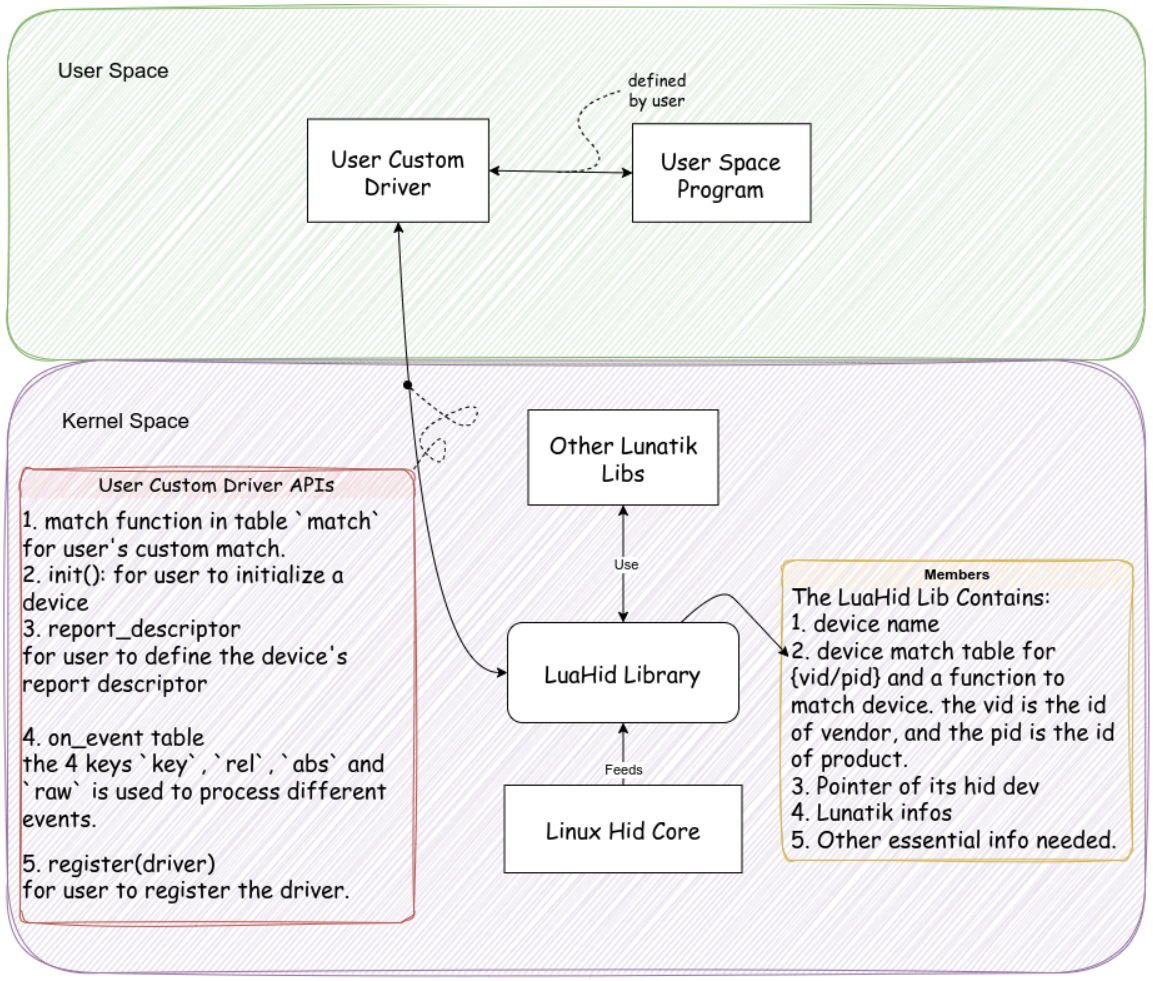


Figure 1: System Architecture

In Figure 1, I define the system architecture, APIs and members of LuaHid library.

From the perspective of global-local coordination, it's a milestone that will establish a dummy framework to validate the architectural integrity of the system. This foundational step ensures alignment between high-level design principles and modular development, allowing iterative refinement of localized components while maintaining structural coherence.

Regarding the internal structure of LuaHid, it will have a built-in struct, a metaclass, and several APIs including `match function`, `report_descriptor`, and `on_event` table etc. I will introduce them in the Expected Deliverables section. These functions are callbacks that allow users to configure their HID drivers and handle special mechanism.

This diagram illustrates the members and basic architecture of the workflow. About How it runs, I will explain it in the next Figure:

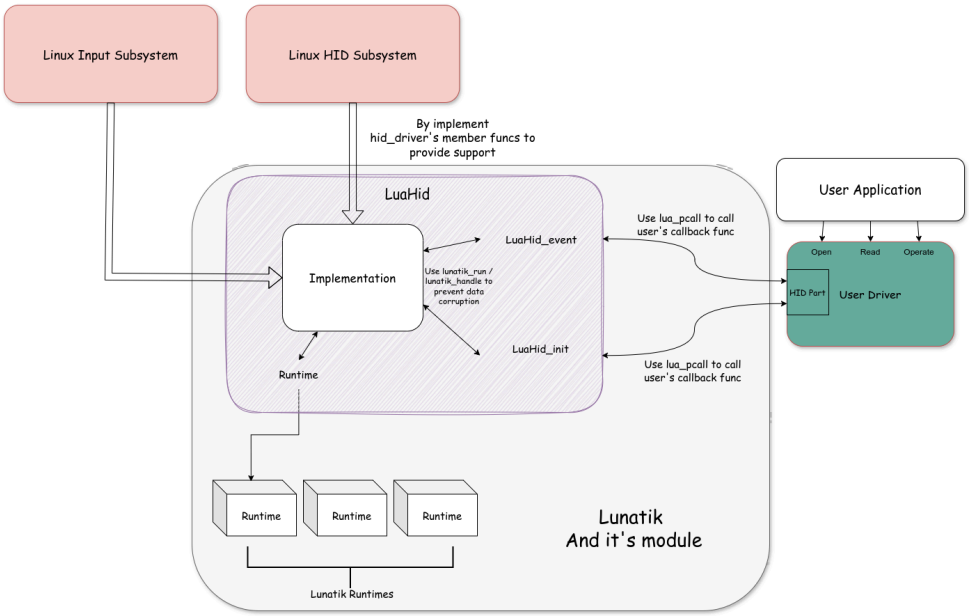


Figure 2: How it runs

In This Figure, I designed how my module interacts with the Lunatik and the Linux HID APIs. It will use Lunatik Runtime for saving essential information and use Linux HID APIs for operates the HID devices. It also shows how to access the device when preventing data corruption and ensure consistency by controlling concurrent access to resources, just by `lunatik_run` / `lunatik_handle` .

2.2.3. API Function Design

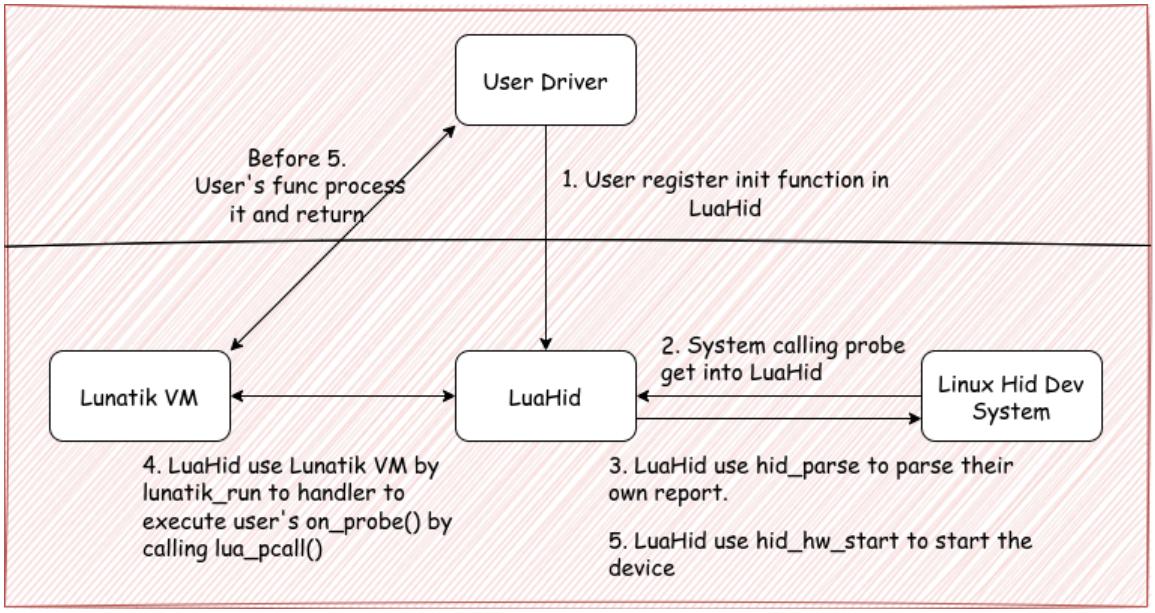


Figure 3: init a driver

In Figure 3, I define the process of creating a driver.

Notably, the `init` function is used to initialize a device.

About release, it will be implemented by LuaHid automatically when the driver is no longer needed. That is because there is no user's custom release functions.

And This APIs is the pattern of APIs which don't use other submodules.

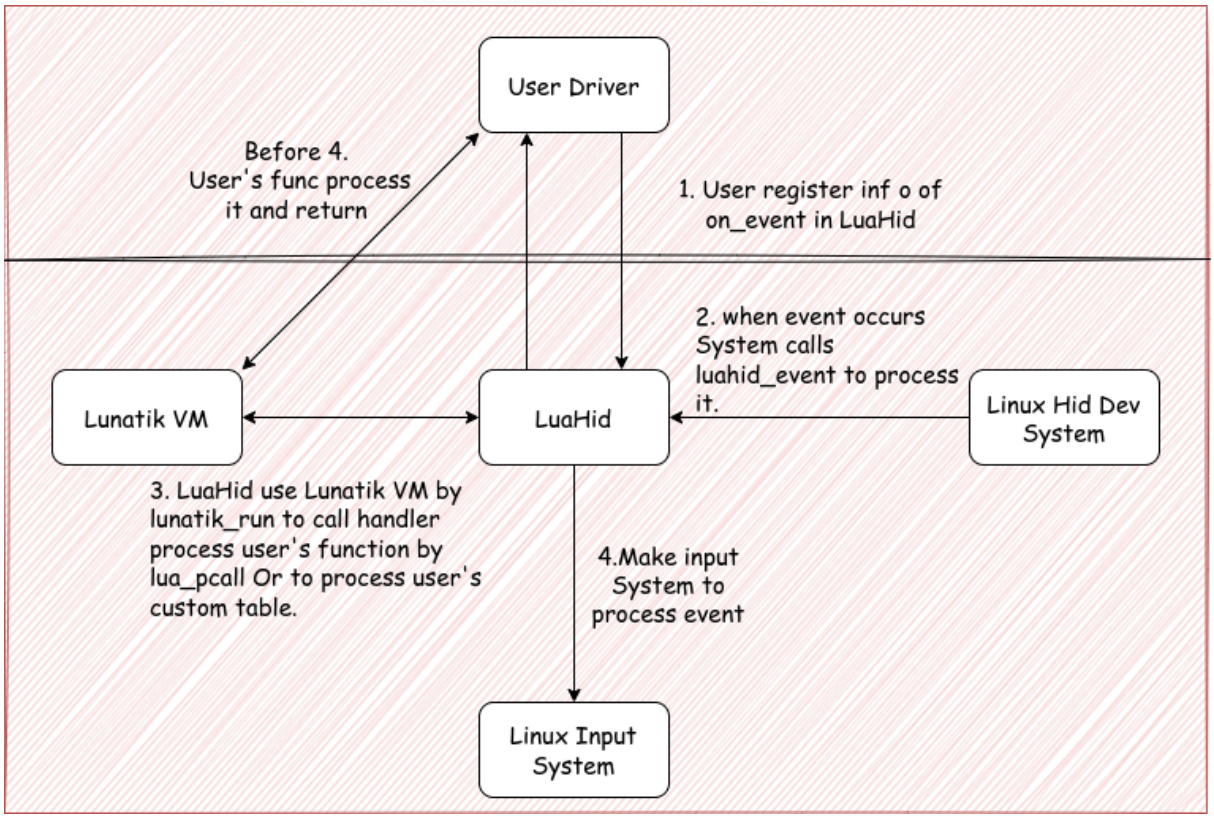


Figure 4: Handle Event

Figure 4 shows how to handle an event.

The table `on_event` is a table consists of several event types, including `key`, `abs`, `rel` and `raw`, each of which is a table or a callback function that contains several event types.

This is also the case where luaHid interacts with different modules such as Linux Input Subsystem. And Other APIs like this will follow the same pattern.

2.3. Expected Deliverables

I will implement the basic APIs and functionalities of the project of the `luahid`, I will list my APIs in the following:

2.3.1. Basic Module `luahid`

1. **hid.register(config)**: Create a new HID driver with specified configuration
   - The config contains `name` as your driver's name, a `match` table used to match the device and a `report_descriptor` table used to describe the report descriptor of the device.
   - The member function of the `config` is the basic APIs of the driver, including `on_event` which contains correspond callbacks and `init` which is used to initialize the driver.
   - The `on_event` is a table contains several event types, including `key`, `abs`, `rel` and `raw`.
     - The `key` is a table contains several key codes, including `KEY.A`, `KEY.B`, etc. The value of the table is declared in `luahid`.

- The `abs` is a table contains several absolute coordinates, including `ABS.X`, `ABS.Y`, and `ABS.Z`. The value of the table is declared in `luahid`.
- The `rel` is a table contains several relative coordinates, including `REL.X`, `REL.Y`, and `REL.Z`. The value of the table is declared in `luahid`.
- The `raw` function is the fundmental of the driver, which gives a stream of data to process the event.
- They are forced to return a bool value to indicate whether the event is handled or not.

Just as the example shows:

```
local driver = {
  -- This is the name of the driver
  -- will be shown in dmesg as the pattern
  -- "[lunatik-hid-abcd]: infos"
  name = "abcd",

  -- This is the match table used to match the device
  match = {
     -- This is the static match table.
     -- given by the "vendor id" and "product id"
     -- identify the device belongs to a certain vendor and a certain productline.
     { vendor = 0x0123, product = 0x4567 },

     -- This function is used to dynamically match the device
     -- the `device` contains the vendor id and product id for user's custom rules. it's used to match a class of devices.
     function(device)
       return device.vendor == 0x0123
     end
  },

  -- This is the table used to describe the report descriptor
  -- which comply with usb hid report descriptor norms
  report_descriptor = {
    0x05, 0x01, 0x09, 0x01, 0xa1, 0x01,
    0x05, 0x09, 0x19, 0x01, 0x29, 0x08,
    -- ... etc
  },

  -- This is the function used to initialize the driver.
  init = function(device)
    -- do something to init the driver
  end,

  -- This is the function used to produce the basic event which is special for the driver.
  -- The undefined event will be handled by the default event handler.
  on_event = {
     key = {
        [hid.input.KEY.A] = function(value)
           print("unsupported A, filtered", value)
           return true
        end,
        [hid.input.KEY.B] = function(value)
           print("Key B occured:", value)
           return false
        end,

        default = function(key, value)
           print("unknown key, filtered", key.name, value)
```

```
            return true
        end
    },
    -- there is no ABS.Z in the example,
    -- so the default event handler of it will be used.
    -- you can also redefine the default event handler.
    abs = {
        [hid.input.ABS.X] = function(value)
            print("position X:", value)
            return true
        end,
        [hid.input.ABS.Y] = function(value)
            print("position Y:", value)
            return true
        end
    },

    raw = function(data)
        if data[1] == 0x01 then
          print("find a special data:", data[2])
          return true
        end
        return false
    end
  }
}

hid.register(driver)
```

2.3.2. Input Module `luahid.input`

The `luahid.input` module is a subset of tools used to handle input events. It contains several tool functions to help users to handle input events. It also contains several `FLAGS` to help users to handle input events.

The entrance of the submodule in `config` is defined like this:

```
local driver = {
 -- other configs
 input = {
   -- This is the function used to remap the input which is special for the driver.
   -- The undefined event will be handled by the default event handler. (just by return nil)
   -- The `mapping` function is used to map a key to a key code before the input system
   -- and the `mapped` function is used to map a key to a key code after the input system.
   mapping = function(usage)
     return hid.input.map_key(hid.input.KEY.KEY_A)
   end,
   mapped = function(usage)
     return hid.input.map_key(hid.input.KEY.KEY_B)
   end,
 }
}
```

I will support those flags in the `luahid.input` module:

1. **hid.input.KEY**: A table contains several key codes, ranging from `KEY_ESC` to `KEY_BTN_TRIGGER_HAPPY40` in `input-event-codes.h`. The value of the table is declared in `luahid`.

The **hid.input.KEYID** is a table numbered by actual key code, which is used to get the key name.

usage example:

```
local hid = require("luahid")
local input = hid.input

-- This is the key code of the key A
local keys = input.KEY
local keyid = input.KEYID

print("The id of Key A is:", keys.KEY_A)
print("The 30th Key is:", keyid[30])
```

2. **hid.input.REL**: A table contains several relative codes, ranging from `REL_X` to `REL_HWHEEL_HI_RES` in `input-event-codes.h`. The value of the table is declared in `luahid`. The **hid.input.RELID** is a table numbered by actual relative code, which is used to get the relative name.
3. **hid.input.ABS**: A table contains several absolute codes, ranging from `ABS_X` to `ABS_MT_TOOL_Y` in `input-event-codes.h`. The value of the table is declared in `luahid`. The **hid.input.ABSID** is a table numbered by actual absolute code, which is used to get the absolute name.
4. **hid.input.EV**: A table contains several event types, including `EV_KEY`, `EV_REL`, `EV_ABS`, and `EV_SYN`. The **hid.input.EVID** is a table numbered by actual event type, which is used to get the event name.

The usage of the `hid.input.REL` and `hid.input.ABS` is similar to `hid.input.KEY`. The example of **hid.input.EV** is shown in the example of `input_event`.

I will support those functions in the `luahid.input` module:

1. **input_event(type, code, value)**: Create an input event with specified parameters. The `type` is the event type (e.g., EV_KEY, EV_REL, EV_ABS), the `code` is the event code specific to the type, and the `value` is the event value. Returns a table representing the input event that can be used with other luahid functions.

usage example:

```
local hid = require("luahid")
local input = hid.input

-- Use these events in a custom handler
local driver = {
 -- ... other configs
 on_event = {
  raw = function(data)
   if matches_condition(data) then
    usage = user_custom_usage(data)
    input.input_event(input.EV_KEY, usage.code, 1)
    return true
   end
   return false
  end
```

```
 }
}
hid.register(driver)
```

1. **map_key(key_name, clear)**: Map a key named `key_name` to a key code. The `clear` is a boolean value, which indicates whether to clear the key mapping or not. The default value is `false`.
2. **map_rel(rel_name)**: Map a relative named `rel_name` to a relative code.
3. **map_abs(abs_name)**: Map an absolute named `abs_name` to an absolute code.
4. **map_other(event_id, event_name)**: Map an event named `event_name` to an event id. The `event_id` is a number, beside key, rel, and abs event, it's given in the evtable.

The `event_name` is a int number, which is defined in `input-event-codes.h` or needs to be defined by users.

usage example:

```
local hid = require("luahid")
local input = hid.input

local keys = input.KEY
local rels = input.REL

function mymapping(usage)
  if usage.hid == 0x12345678 then
    return hid.input.map_rel("HWHEEL")
  elseif usage.code == keys.BTN_1 then
    return hid.input.map_key(keys.BTN_2)
  elseif usage.code == keys.BTN_2 then
    return hid.input.map_key(keys.BTN_1)
  end
end

local driver = {
  -- ... other configs
  input = {
    mapping = mymapping,
  }
}
hid.register(driver)
```

2.3.3. Other Useful functions

1. **hid.send_report(device, report, isset)**: Send a report to the device. The `device` is the device name, the `report` is the report data, and the `isset` is a boolean value, which indicates whether to set the report or not. The default value is `false`, means to get the report.

   usage example:

   ```
   local hid = require("luahid")
   local input = hid.input

   -- This is the report data
   local init_report = {
     0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
   ```

```
  -- ... etc
}
-- This is the device name
local device = {
  -- some config

  init = function(device)
  -- Send a report to the device
    hid.send_report(device, init_report, true)
  end,
}
hid.register(device)
```

2.3.4. Plan to Port a HID Driver

I will port a `Xiaomi` mouse driver to Lua & Lunatik, and i will port some `logitech` mouse drivers to Lua & Lunatik including Logitech `B330` . About the keyboards, i will port a `Logitech` keyboard driver to Lua & Lunatik including `K380` .

When it comes to debuggability, i think port a virtual HID driver is a good choice. I will implement by C and port a simple virtual HID driver to Lua & Lunatik, in order to enhance the debuggability of the project.

2.3.5. Benchmarks

Benchmark comparison between the original HID drivers and their Lua implementation:

1. **Test Criteria:**
   1. **User Impact:**
      - Measure the average event processing delay for both implementations.
      - Monitor CPU usage and memory overhead under high-frequency event loads.
      - Evaluate the impact on interactive scenarios to ensure the Lua driver does not degrade user experience and system processing.
   2. **Development Efficiency:**
      - Compare the complexity by lines of code in ported drivers(from `Xiaomi` and `Logitech` i have mentioned above).
      - Collect qualitative feedback on code readability, ease of debugging, and maintainability, by LOCs, count of functions and other metrics.

2. **Test Environment:**
   - Use identical hardware and OS configurations.
   - Also Use virtual machine such as `Qemu` to run the tests.
   - Run baseline tests on Lua-based implementation in Lunatik.

3. **Test Methodology:**
   - About the time to evaluate how our lua driver will impact Linux kernel performance, i will measure the ratio of the time taken by the Lua driver to the time taken by the original C driver. And i also will measure the time taken by the Lua driver to process events and the time taken by the entrie hid event workflow.
   - About the time to evaluate how the hid driver will impact the user experience, i will measure the ratio of time taken by the Lua driver divided by the Max Polling Interval which is defined in [1] as `bInterval * 125 µs` of the device.

This kind of metrics is used in research papers to evaluate the performance of the HID driver. [2]. If lua driver takes more time than the Max Polling Interval, it will blocked next event and influence the user experience. This It will be different for different devices and different scenarios.

2.4. Timeline and Milestones

There is a timeline and milestones for my project as a table:

Period	Milestones	Done?
Pre GSoC Period	• I have finished a simple task adding linux time for lunatik in my fork. • I have written a small HID driver in C for my Logitech mouse. click me° • I have read part of the Lunatik source code and the Linux HID driver source code. • I have a communication with the community about the basic design of the project.	Done
May 8th — June 1st (Community Bonding Period)	The main goal of this period is to adding APIs and revise the design of the project according to the feedback from the community in order to align with real HID drivers in Linux. • Revise design based on community feedback, including the APIs and the design itself of the project with widely discussion. • Study the usbhid drivers in Linux and the HID low-level APIs in Linux. • Read the Lunatik source code to understand the design of Lunatik and the Lua C API. • Learning Lua C API and lunatik itself.	No

June 2nd — June 30th	The main goal of this period is to implement the basic APIs and the basic functions of the project as follows: • function `hid.register()`, and related APIs including `match`, `name`, the `raw` component of `on_event`, and the `report_descriptor` field in `config`. • Write example scripts for the basic functions to test it. • About my **PRs**, I will submit about 3 PRs to finish the basic APIs and the basic functions of the project. ▸ a PR for Update the Makefile, and the basic framework this project. ▸ a PR for the `hid.register()` function, the `match` table, the `report_descriptor` table and `init` function, which supports user to define the report descriptor to customize the action of the device. that means the device can be supported by the default driver. ▸ a PR for the `on_event` table, which supports user to define the event handler of the device. ▸ a PR for the example lua scripts, which supports user to test the basic functions of the project. (not necessary to be a PR) It is the first milestone of the project. It is a sign of the project's feasibility — The project won't failed.	No
July 1st — July 13rd	The main goal of this period is to implement the the APIs • Implement the `on_event` function in the config, including `key`, `abs`, `rel`, i will create a PR for it. • Port HID drivers from linux into Lua & Lunatik and run it to test the	No

	basic function. i will create a PR for it. • **Important**: preparing to draft evaluation in providing my materials for the mid-term.	
July 14th — July 18 18:00(UTC)	In this period, i will cope with the mid-term evaluation. Focusing in the evaluation and the feedback from my mentor about the work i will have done.	**No**
July 19 — August 2nd	The main goal of this period is to implement the all APIs I have promised in the proposal and possible extra APIs added in the Community Bonding Period. • Implement all the `hid.input` and `hid.send_report` APIs. • Write runnable example scripts for the all functions. • I will create a PR for the `hid.input` APIs and the `hid.send_report` APIs, and the example scripts. • I will port a `Xiaomi` mouse driver to Lua & Lunatik, and port some `Logitech` mouse drivers to Lua & Lunatik, including the Logitech `B330`, via a pull request (PR). • I will fix bugs and enhance my previous code incrementally by submit PRs. • Begin benchmarking the project.	**No**
August 3rd — August 25th	The main goal of this period is to finish the project and prepare the final report. • **Important**: draft a final report and prepare the final evaluation. • **Important**: submit a PR to the Lunatik project to submit the final version.	**No**

	This period is a little bit relaxing to prevent the project from being delayed.	

3. Post GSoC

If I'm accepted, I will be proud to contribute as a maintainer for the Lunatik project. I plan to support gaming devices extensively and document my experience in a detailed blog post in both Chinese and English.

Bibliography

[1] “USB HID Class Definition.” [Online]. Available: https://www.usb.org/sites/default/files/documents/hid1_11.pdf°

[2] Raphael Wimmer Andreas Schmid and F. Bockes, “On the Latency of USB-Connected Input Devices.” [Online]. Available: https://epub.uni-regensburg.de/40182/1/On_the_Latency_of_USB-Connected_Input_Devices_author_version.pdf°

Index of Figures

Figure 1 System Architecture 6
Figure 2 How it runs 7
Figure 3 init a driver 7
Figure 4 Handle Event 8